

COMPUTER PROGRAMMING

A **Computer program** is an application used to accomplish any task imaginable in soft form.

Basic steps in programming

It is important to take a structured approach when programming thus implementing a program problem in certain order of steps.

1. Plan the program's tasks
2. Design the user interface
3. Write the program's code
4. Test and debug the program (using if possible alpha and beta testing methods)
5. Document and distribute the program.

Type of common programs

- Packaged programs they can be purchased from software stores, ordered via email and direct from manufacturer.
- Costumed programs they are designed for a particular or specific purpose within a given organization.

STARTING VISUAL BASIC

Create anew project, which is a set of files that store information about the components of an application program.

1. Start visual basic
2. Choose file, new project (if new project dialog box does not occur)
3. Select or specify the type of project you want to create e.g. **Standard. exe** option
4. Then click okay and you will be placed in visual basic's design environment

If the title bar is specified as design, it means that you are in design mode (design time) and you can design and code the program application.

When the program is running and the title bar is named as run mode; this means that the program is running or executing.

Visual basic applications are comprised of one or more components such as

- Forms
- Coded modules
- Classes
- Controls
- Other components

Visual basic has the Project explorer, which consists of a list of the contents of the current project. It grows as many components are added.

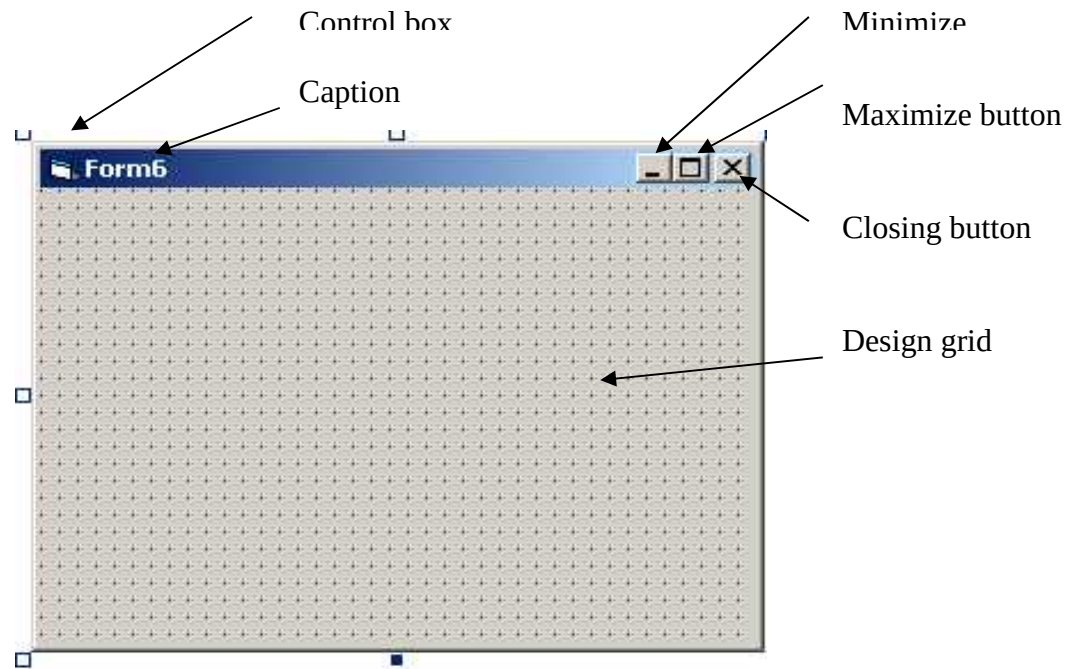
RUNNING PROGRAM

To run the program you need to execute visual basic start command using any of the methods

- Click the start button on visual basic toolbar
- Choose run, start
- Press F5

Forms

A **Form** is a container that holds all the other controls such as labels, textboxes, pictures and access databases and make up part of the user interface.



Forms have features found on a window in a program. It contains a title bar, a control menu, and asset of minimize/maximize/restore, and close buttons.

Design time features are:

Close buttons

- They are always present even if the properties are set in such away that they are not visible at runtime.

Design grid

- It is a grid of dots that allows you to line up controls easily, as you are designing your interface.

The form oversees the use of objects that make up the interface, including storing program codes that relates to the objects on the form and respond to events.

Some form properties

Property Name	Description
border style	Sets the type of border that is used for the form
Control box	Determines whether the controlbox(containing the move and close menus) is visible when the program is running.
font	Determines the font used to display text on the form
icon	Determines the icon that is shown in the form's title bar and that appears when the form is minimized.
Maxbutton	Determines whether the maximize button is displayed on the form when the program is running
Mdichild	Determines whether the form is a child form for an MDI application
minbutton	Determines whether the minimize button is displayed on the form when the

	program is running.
startup position	Determines the initial position of a form when it is first shown.
window state	Determines whether the form is shown maximized, minimized, or in its normal state.

Controls

Visual basic controls let you perform a wide variety of tasks including editing text, displaying pictures, and interfacing with a database. The liberal use of controls has always been one of VB strongest features.

Controls are objects that perform specific tasks and have associated properties, events and methods.

E.g. using a text box control, you can set properties to determine the size of the text box, the text that it displays and the color of the text and background.

You are not limited to using only the control s provided by Microsoft, the design allows easy integration of third-party controls which increases chances of finding a control to perform almost any task you want, from data acquisition to custom reporting to specialized graphics processing to game play, and everything in between. You can create your own activex control for use in your programs.

N/B

A control is a miniature program in itself that perform certain tasks for you e.g. a textbox contain internal code that allows it to process keystrokes so that it knows, characters when you press back space key.

Control properties

They are adjectives that describe one specific thing about an object and modify objects appearance when you run your program. Hence a windows appearance is defined by a set of properties thus they are settings that control the appearance and behaviour of an object. All objects do not have it set of properties although we have common properties found in many objects.

Common properties

Name, Index, Left, Top, Height, Width, Enabled, Visible, Caption

Left and Top

- Describe the position of the form on the screen

Width and height

- Define the size of the form on the screen

Caption

- Dictates what is shown in an object title bar

Name

- Describes a unique identifier by which you can refer to the object in code. {name property is very important , in that it identifies the control since the program may have many controls of the same type and you can also use the control name property to indicate the procedure in which a particular code statements written.}

Visible

- Determines whether an object can be seen on the screen by setting it true (to be viewed) and false otherwise.

Enabled

- Determines whether the users can interact with an object setting it either to true or false.

Control methods

Is a program function that is built into the object. As properties, they are referenced using code via the dot notation

E.g. form1.printform-prints an exact image of the form named form1 on the line printer

Although different objects have different methods, many objects have the same or common methods.

Like

Drag

- This handles the operation of the users' dragging and dropping the object within its container.

Move

- Changes the position of an object

Set focus

- Gives focus to the specified control

Zorder

- Determines whether an object appears in front of or behind other objects in its container.

Controls responding to action with events

Objects respond to actions, whether generated by the users or externally. Response to actions are handled through use of events. Generally an object is the set of events it can raise and the user actions that trigger them.

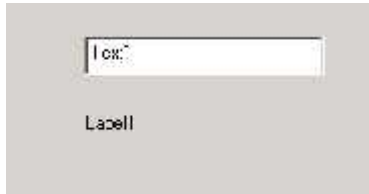
Intrinsic Controls

CONTROL	DESCRIPTION
picture box	Displays graphics. Can also serve as a container for other controls.
Label	Displays text that the user
Text box	Displays text. also allows the user to enter and edit the text.
Frame	Serves as a container for other controls. Provides grouping of controls
Command button	Allows the user to initiate actions by the button.
Check box	Lets the user make the a true /false choice
Option button	Lets the user choose one option from a group of items.
Box	Lets the user choose from a list of items or enter a new value.
List box	Lets the user choose from a list of items.
Horizontal scrollbar	Lets the user choose a scrollbar value based on the position of the button in the bar.
Vertical scrollbar	Serves as a horizontal scrollbar.
Timer	Lets the program perform functions on timed basis.
Drive list box	Lets the user select disk drive
Directory list	Lets the user select a box directory or folder
File list box	Lets the user select a file
Shape	Displays a shape in time form
Line	Displays a line on the form
Image	Similar to a picture box control. uses fewer system resources but doesn't support as many properties, events and methods..
Data control	Provides an interface between the program and a data source
Ole	Provides a connection between the program and an OLE server.

Common dialog	Allows user of windows standard dialog boxes to retrieve information such as file names, fonts and colors.
Label control	It does not allow the user to enter text directly; it contains events and properties that allow the programmer to manipulate the text directly from code. E.g. label caption="time is"
Textbox	It has property that makes programs and user interaction versatile. Multiline and scrollbar properties in the textbox windows. They are useful for handling large amounts of text. Scrollbar property has affect on the textbox only if its multiline property is set to true

Controls for displaying text

Textbox and label



Textbox

Allow both data/text display and also allow text input. A text box control must be adding to a form so that it can be used.

The textbox control is found in the tool box as one of the default controls on toolbox

Steps followed to use a text box

1. Click the tool for the textbox control in the toolbox. (To know if it is a text box let your mouse pointer pause over each control in turn to see a tooltip.)
2. Move the mouse pointer to the form .Note that the pointer will change into across hair, indicating that you're about to draw the control.
3. Move the pointer to one corner of the area where you want to draw the control.
4. Click and hold the left mouse button
5. Drag the mouse to where you want diagonally opposite the corner of the control.

Label

Are designed only to display text. They inform users what type of information is to be entered in the textboxes if laced next to input field. It has events and properties that allow the programmer to manipulate the text directly from code. If you want to display more text, pay attention to the auto size and word wrap properties. If auto size is true the size of the label control automatically adjusts to fit the text being displayed, otherwise label remains same size.

If word wrap is false the label expands horizontally to a low the caption to fit, even if the label grows so large that it passes the right edge of the screen. If the word wrap is true the label expands vertically to allow enough times of text to accommodate the caption even if the label runs off the bottom edge of the screen.

Steps followed to use a label

1. Select the label control's tool in the tool box
2. Draw a label control to the left of the first text box control
3. In the properties window change the label control's name property to start with lbl e.g. lblbookid
4. Change the label control's caption property to <name>
 - i. lblbookid.caption=<name>

5. change the label's alignment property to suit your application
 e.g. lblbookid.alignment = 0-leftjustify
 1-rightjustify
 2-center

The command button



This control lets users initiate actions by clicking the button. you can set up a command button control by

- o Drawing the button on the form
- o Then set the caption property to the text that you want displayed on the button's face.
- o To activate the button, just place code in the button's click event procedure.

To access commands through the keyboard

You make use of access keys assigned using the ampersand sign "&" ,on the button caption. To use the access key, the user holds down the alt key and processes the access key to trigger the command button controls' a click event.

e.g. command1.caption=&"print"

It will appear as Print and to use the command using the keyboard press alt + p

Checkboxes



when a checkbox is on ,a check mark(✓) is displayed in the box.It indicates that the answer to the checkbox 's corresponding question is "yes" .when the checkbox is off or unchecked the box is empty, indicates an answer of "NO"

option buttons-they exist a group and only one of them can be selected at a time .They are useful for presenting a fixed list of mutually exclusive choices.

Two ways of using the option button.

- 1.) Use the click event if you want to take an action when users select an option. This is useful when using control array and code is typed in the option procedure.
- 2.) Do not write any code in the optional button event .Instead use if statement in a relevant controls procedure e.g. button to check if they are true .

The list box

-The key parts of the list box are the following

Item list

- o Which is the list of items from which the users can select.

- o They are added to the list in the design environment or by our program as it is running mostly using code

Select item

This item is chosen by the users. Depending on the style of list you choose a selected item indicated by a highlight bar or by a check in the box next to the item.

Scroll bar

This part indicates that more items are available on the list than will fit in the box and provides the users with an easy way to view the additional items. , But it does not have horizontal scrollbar although vertical scrollbar appear as default.

Filling list box using code

```
Private sub form_load()
    Dim I as integer
    For I=1 to 100
        List1.AddItem I
    Next I
```

To clear using clear method

```
List1.clear
```

To print items from the list you can use list property of the list box indexed from “ 0”

```
Print list1.list (index)
```

Listcount-represent the number of items in a listbox

```
List1.listcount
```

Listindex- represent index of selected items.

```
List1.ListIndex
```

Removeitem-remove item from the item list by specifying the array index of the item.this property works only for a single selection.

```
List1.RemoveItem ListIndex
```

Using index with additem method you can specify the location within the list where you want the new item to appear.

Setting the appearance of a list box

This can be done by using or setting the style property to either:

0-standard

1-checkbox

the values can be changed at runtime by setting its value to one of the intrinsic constants

vblistboxstandard or vblistboxcheckbox respectively

the other way to change the list's APPEARANCE is to use the columns property that has a default value of zero

Setting it to 1

Causes the list to be presented with one column at a time, but to scroll horizontally instead of vertically.

Setting it to values greater than 1

Causes the list to display in the number of columns specified by the property.

Working with multiple selections

Applicable only when you need to select more than one element .this is done using multiselect property and it has three possible setting.

1. 0-none
Is default setting showing that multiple selection is not required or permitted.
2. 1-simple
Users can click an item with the mouse to select it or click a selected item to deselect it. Using the keyboard, users can make use of the cursor keys to move the focus (dotted line border)to an item and then press the space bar to select or deselect it.
3. 2-extended
Is more complex, here users can use standard windows techniques to quickly select multiple items.you can select a range of items by clicking the first item in the range and then ,while holding down the shift key,click the last item in the range; all items in between the first and last items are selected .to add or delete a single item to or from the selection,users hold down the control key while clicking the item.

Selected property is an array that has an element for each item in the list. The value of the selected property for each item is either true (if the item is selected) or false (when not selected). To display item selected you can print on the form

e.g.

```
numitem=list1.listcount
```

```
For I=0 to numitem-1
```

```
    If list1.selected (I) then
```

```
        Form1.print list1.list (I)
```

```
    End if
```

```
Next I
```

The combo box

- The use of combo boxes can be done in three different ways.



- 1.) The drop-down combo box

Presents the users with a text box combined with a drop down list. The users can either select an item from the list portion or type an item in the text box portion

- 2.) The simple combo box

Displays a textbox and a list that doesn't drop down.

- 3.) The drop down list

Displays drops down list box from which the users can make a choice. The users cannot enter items that not in the list. As a list box it uses AddItem, RemoveItem, and clear methods to modify the contents of the list, but combo box mainly lacks support for multiple selections, although it allows you to enter choices

Filling combo box using code

```
Private sub form_load ()  
    Dim I as integer  
    For I=1 to 100  
        Combo1.AddItem I  
    Next I
```

To clear using clear method

```
combo1.clear
```

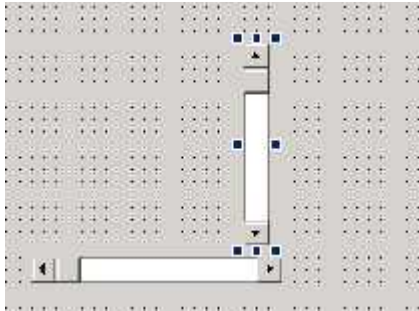
To print items from the combo list you can use list property of the combo box that is indexed from “ 0”

```
Print combo1.list (index)
```

Special purpose controls

- Scrollbar control
- Timer control
- Frame control

SCROLLBAR CONTROL



Provides a graphical way for you to perform tasks such as control your position within a document or make adjustments. Visual Basic has two types of scrollbars for entering numerical data

The vertical scrollbar –vScrollBar

The horizontal scrollbar –HScrollBar

Setting up the scrollbar

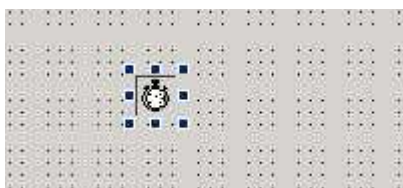
To use a scrollbar, you simply use appropriate properties to set and retrieve the range and values. The three most important properties used with scroll bars both at design time and runtime.

Value-retrieves or sets the current numeric value

Min-controls the minimum value of the value property

Max –controls the maximum value of the value property

THE TIMER CONTROL



The timer usually executes code when the interval is complete. The timer control counts down repeatedly, as long as the enabled property is set to true. The timer control is designed to work with very small amounts of time; the maximum setting is just a little longer than a minute.

The timer control is used for scheduling and performing repeated operation.

To Setting up timer

Draw it on the form

Always shows up as an icon at design time, but it does not show up at all while your program is running.

Place the code for events generation in the timer event and then

To make use of timer

Set the interval property

It is set to zero by default disabling the timer control. This property can be set to any value between zero to 65535, with the amount of time in milliseconds.

Set the enabled property to true

It acts like a switch that turns your timer on and off. If the timer control is enabled, the code in the timer event is executed at the end of the time specified in the interval property.

THE FRAME CONTROL

Is a special type of control called a container. Containers are controls in which you can draw other controls, much like a real picture frame visually encloses (or contains) an artist's picture. At this point, you can set several properties that control the appearance of the frame.

e.g.

caption-the text that is entered in the caption property will appear in the upper-left corner of the frame. Is used to identify the contents of the frame or provide other descriptive information.

Appearance-declares whether the border is shown as a single-line, single color border, which gives the control a flat look, or a 3D effect.

BorderStyle-this property determines whether the border around the frame is displayed.

MENUS

Menu system can be created using the menu editor. You can use this editor to create a menu bar for each of your application's form as well as context-sensitive "pop-up menus {menu system enable a program user to locate and execute its functions easily}

To create menu, first open the form on which you want the menu located, and then start the menu editor in any of the three ways

Click the menu editor button on the tool bar

- ♣ Choose tools, menu editor
- ♣ Press
- ♣ Control +E

You use the ampersand sign next to the menu caption first character, which is to become the access key.

Access keys offer users a way to navigate a menu system by using the keyboard instead of a menu. If a menu item has an access key defined, it is indicated by an underscore beneath the letter in the item's caption.

Shortcut keys provide direct access to a function through a single key (such as delete) or key combination (such as control+s), bypassing the menu system entirely.

To assign a short cut key

- 1.) Enter the menu editor, select the menu item for which you want a short cut key.
- 2.) Then select the desired key from the short cut list.

The key is assigned to that function, and the short cut key information appears next to the menu item in the menu.

Generally assigning shortcut key

- Select form and then open the menu editor
- Click the print menu item to select it
- Click the drop down arrow next to the short cut list and select control + p from the list
- Click ok to close

WRITING CODE

Visual basic is an object oriented, event driven language. The user initiates events, by writing code that respond to those events appropriately, in the object event procedure. Procedure or sub procedure is a discrete sequence of code statements that has a name and is executed as a unit.

You can double click

To get code view

You can double_click the controls to get the control event procedure

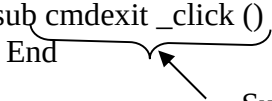
Click code view icon in the project explorer

Highlight menu view then in the drop down menu highlight code view

Then type code

e.g . In exit procedure control

```
Private sub cmdexit_click ()
End
End sub
```



Sub procedure

To indent the code you press tab: Indenting code makes it easier to read and trace the code

A comment is a line that explains what's going on, a comment is denoted by a single quotation mark ('), usually at the beginning of a line. When the visual basic interpreter encounters a single quote on a line, the remainder of the line is ignored. Visual basic moves to interpret the next line for next instruction.

Visual basic continuation character (_) will be used at the end of the line that is broken.

Although you can leave this character out and type the code in a single line.

Keyboard hotkeys are indicated with underlining .you use ALT + <CHARACTER TO BE UNDERLINED> to select the menu item

In the code view /window we have two drop down list boxes at the top

The object box (one on the left) lists all the objects that have been placed on the current form, as well as the form itself.

The procedure box (on the right) lists all the events that apply to whichever object that is currently selected in the object box.

Writing program code

Initialize or declare variables

A variable is a temporary storage location for information or unique identifier of a given type.

Declaration

The first part of the program/procedures is used to declare the variables you'll need .telling visual basic the variables that the procedure will be using ,as well as what type of information each variable may contain.

Procedure code is part of code remaining in the procedure that do not work.

Declaration of variables

Visual basic supports two types of declaring variables.

Explicit

Implicit

Explicit

In this type of declaration you make use of statements to define the names and data types of your program's variables. The statements do not assign values to the variables but merely tell visual basic what the variables should be called and what data they can contain.

e.g.

```
Dim varname [as vartype]
Private Varname [as vartype]
Public varname [as vartype]
Static varname [as varitype]
```

Implicit

You can assign a type as in the declaration above but in this type of declaration a special character is used at the end of the variable name when the variable is first assigned a value.

DATA TYPE	CHARACTER
Integer	%
Long	&
single	!
double	#
currency	@
string	\$

The above data types are the only type that have special characters the rest have none.

e.g.

```
explicit    nNumVal=0    sStr="my name"    fNum=10.1
implicit    nNumVal%=0    sStr$="my name"    fNum! =10.1
```

Fixed length strings

Most string types that are used in visual basic are of variable length. They contain any amount of text up to 2 billion characters and can be declared in both implicit and explicit declaration.

Dynamic string variables usually adjust automatically in size to accommodate the length of the string.

In fixed length strings, string variable size remain the same, regardless of the text assignment. so the expression assigned to the variable should be shorter than the defined length of the variables, and the remaining length of the variable is filled with spaces. if the expression is longer only the assigned length that fits the length will be stored while the rest of the text will be truncated. fixed length string variables can only be declared using an explicit declaration.

Variable arrays

Arrays are normally declared as

```
<Varname> (size) as [vartype]
e.g. Dim nData (1 to 20) as integer
```

But variable array declaration will be declared as

<Vname> () as [vartype]
 e.g. Dim nData () as integer
 sname="people"
 Redim nData (1 to len (sname))

Public variables are global thus they can be accessed anywhere in the program
 Dim/private variables are localized variables hence they are used within the scope of the procedure or form. Thus they are known as form level or module _level variables

Static variables retain the current value of variables within a running procedure. Normally variables are discarded by visual basic when the procedure is finished, but when you need to call the procedure several times you need to preserve the value of a variable even after the procedure is run and the variable for one call to the procedure is dependent on the value left over from previous calls

e.g. static npagenunder as integer

Constant variables are declared at the beginning of the form program or procedure and they constant variable store the value permanently and they can be used within the level of declaration without changing in value.

Const <varname>=value
 e.g. Public const pie=3.142

TYPES	PREFIX	RANGE	MEMORY REQ	
Integers	n	-31768 to 32767	2 bytes	Whole numbers
Long	l	Approximately +/- 2 billion	4 bytes	
Byte	B	0-255	1 byte	
Single (float)	f	+/- (1E-45 to 3E38)	8 byte	Decimal numbers
Double	d	+/- (5E-324 to 1.8E308)	8 byte	
Currency	c	+/- 9E14	8 byte	Numbers with up to 15 digits left of the decimal and four digits right of the decimal
String	s	65400char for fixed length string and 2 billion for dynamic string	1 byte per character	Text information
Date	D	1/1/100 to 31/12/9999	8 byte	Date and time information
Boolean	b	True/false	2 byte	Logical values
Object	o	Instances of class; ole objects	4 byte	
Variant	v	Accepts any of the above or preceding data types	16 bytes	

Visual basic program constructs

Sequences

Visual basic program allows the programmer to place the code in a given set or order of execution.

Example

```
Print "hallo world"
Print "we want peace"
```

Selection

It uses selection constructs to make choices

Example

If then

Syntax

```
If <condition> then
    (statement)
    .
    .
    .
end if
```

example

```
if (Not adodc1.recordset.eof)=true then
    print adodc1.recordset.fields(0).value
    adodc1.recordset.movenext
end if
```

if then else

syntax

```
if <condition> then
    statement (if condition is true)
else
    statement (if condition is false)
end if
```

example

```
if k<=100 then
    print "iam within the limit"
else
    print "iam over board"
end if
```

Nesting If Statement

Method 1

```
if <condition1> then
    if <condition2> then
        statement1
    end if
    statement 2
else
    if <condition3> then
```

```

        statement3
    else
        statement 4
    end if
end if

```

Method2

```

if <conditon1> then
    statement1
elseif<condition2>then
    statement2
    elseif<condition3> then
        statement3
end if

```

ITERATIONS

Loops continuously in accordance with the loop condition statement status

Do while

Syntax

```

Do while <condition>
    Statement
Loop

```

Example

```

Do while I<20
    List1.Additem I
    I=I+1
Loop

```

Do until

Syntax

```

Do until<condition>
    Statement
Loop

```

Example

```

Do until j>19
    Combo1.additem j
    J=j+1
Loop

```

Do loop while

Syntax

```

Do
    Statement
Loop while<condition>

```

Example

```

Do
    Print "new number" & k

```



```
K=k+1
Loop while k<100
```

For to do

Syntax

```
For varname=initialvalue to maxvalue
    Statement
Next varname
```

Example

```
For l=0 to 20
    T(l)=0
Next l
```

While do

Syntax

```
While <condition>
    Statement
Wend
```

Example

```
While rsmine.eof=false
    Print rsmine.fields(0).value
Wend
```

The with statement

Syntax

```
With statement
    statement
End with
```

Example

```
With adodc1.recordset
    !delete(adaffectedcurrent)
end with
```

Math operators

OPERATION	SIGN
Addition	+
Subtraction	-
Multiplication	*
Division	/
Integer division	\
Modulus	Mod
Exponentiation	^

String functions

Visual basic supports only one string operator the combination or concatenation Operator that is the ampersand sign (&). It joins two string or a string and a value

syntax

new string=string1 & string 2

example

sk ="my name" & " is my country"

Visual basic string function

Lcase

It converts all text to lower case

Syntax

Lcase (string)

Example

Lcase ("KENYA MY COUNTRY")

'Returns "Kenya my country"

Ucase

It converts all text to upper case

Syntax

Ucase (string)

Example

Ucase ("hallo Kenya")

'Returns 'HALLO KENYA"

Instr/InstrRev

Returns the location value of a search string inside another string

Instr

Syntax

Instr (startingpt, sourcestr, searchstr, casemode)

0----case sensitive

1----non case sensitive

vbbinarycompare

Example

Instr(1,"together","e",vbtextcompare)

'returns 4

InstrRev

Syntax

InstrRev(sourcestr,searchstr,startingpt,casemode)

Example

InstrRev("Example","a",-1,1)

'returns 3

Len

Returns the size or length of the string

Syntax

Len(string)

Example

```
Len("mypeople")
,returns 8
```

Trim

Removes spaces from the whole string

Syntax

```
Trim(string)
```

Example

```
Sp="          my country my people          "
Trim(sp)
'returns "my country my people"
```

Rtrim

Removes spaces from the right side of the string

Syntax

```
Rtrim(string)
```

Example

```
Rtrim(sp)
'returns "          my country my people"
```

Ltrim

Removes spaces from the left part of the string

Syntax

```
Ltrim(string)
```

Example

```
Ltrim(sp)
'returns "my country my people          "
```

Left

Retrieves a specific amount of characters from the left part of the string

Syntax

```
Left(string,numchar)
```

Example

```
Left("application",3)
'returns "app"
```

Right

Retrieve or specifies a specific number of characters from the right part of the string

Syntax

```
Right(string,numchar)
```

Example

```
Right("application",3)
'returns "ion"
```

Mid

Retrieves or replaces the specified amount of text in a string

Syntax

```
mid(sourcestr,[startingpt,numchar])
```

Retrieves

```
Svalue=mid$("today",2,3)
'returns "oda"
```

replaces

```
mid$("today",2,3)="XXX"
```

‘returns “txxy”

StrReverse

reverses the order of characters in a string

syntax

strReverse(string)

example

strReverse(“yelling”)

‘Returns “gnilley”

Chr

converts ASCII numbers(0 to 255) to related ASCII letter,alphanumeric,and numbers

syntax

chr(value)

example

chr(65)

‘returns “A”

Asc

The inverse of Chr() converts ASCII letters,alphanumerics and numbers into number values.

Syntax

Asc(string)

Example

Asc(“a”)

‘returns 97

asc(“ ”)

‘returns 32

CStr/Str

Convert values or variables value into strings

Syntax

Str(value)

Example

Str(89)

‘returns “89”

Val

converts strings into double value or number

syntax

Val(string)

Example

Val(“1200”)

‘returns 1200

val(“4nm00”)

‘returns 400

val(“”) or val(“ty”)

‘returns 0

Cint

Converts string s into integer value

Syntax

Cint (string)

Example

Cint(“1200”)

Taking values from the combo box after clicking the combo list

Assigning relevant textboxes with values according to what is resultant from the string respective string input operation

End Sub

Initializing values and filling the list and combo box

```

Private Sub Form_Load()
    Dim i As Integer
    For i = 65 To 110
        Combo1.AddItem i
    Next i

    i = 1

    Do Until i > 9
        List1.AddItem (i & i & (i + 1) & (i + 2))
        i = i + 1
    Loop

    Text19.Text = 1
    Text20.Text = 0
    Text21.Text = 1

End Sub

```

Assigning text box with values from the list

```

Private Sub List1_Click()
    If List1.Text <> "" Then
        Text17.Text = Val(List1.Text)
    End If
End Sub

```

Retrieving the left value of the string

```

Private Sub Text10_Change()
    Text4.Text = Left(Text8.Text, Val(Text10.Text))
End Sub

```

Retrieving the right values of the string

```

Private Sub Text11_Change()
    Text5.Text = Right(Text8.Text, Val(Text11.Text))
End Sub

```

Converting value to string and string to integer respectively

```

Private Sub Text17_Change()
    Text15.Text = Str(Val(Text17.Text))
    Text18.Text = CInt(Text17.Text)
End Sub

```

Retrieving values using the mid function

```

Private Sub Text19_Change()
    Text6.Text = Mid(Text8.Text, Val(Text19.Text), Val(Text20.Text))
End Sub

```

```

Private Sub Text20_Change()

```

```
Text6.Text = Mid(Text8.Text, Val(Text19.Text), Val(Text20.Text))
End Sub
```

Return the position of a string within the source string

```
Private Sub Text21_Change()
Text3.Text = InStr(Val(Text21.Text), Text8.Text, Text9.Text, vbTextCompare)
End Sub
```

Convert ascii characters to numbers

```
Private Sub Text22_Change()
Text16.Text = Asc(Text22.Text)
End Sub
```

```
Private Sub Text6_Change()
Dim spill As String
spill = Text8.Text
If Val(Text19.Text) > Len(spill) Then
Mid(spill, 1, 0) = Text6.Text
Else
Mid(spill, Val(Text19.Text), Val(Text20.Text)) = Text6.Text
End If
Text8.Text = spill
End Sub
```

```
Private Sub Text9_Change()
Text3.Text = InStr(Val(Text21.Text), Text8.Text, Text9.Text, vbTextCompare)
End Sub
```

Procedures and functions

Procedures

They are subroutines or segments of code that perform a particular task and then return processing to the area of the code from which they were called .

Advantages

- You can test each task individually (the smaller amount of code enables or makes debugging easier)
- Help eliminate redundant code by calling procedure instead of repeating program code every time.
- You can create a library of procedures to be used in more than one program
- Makes program maintenance easier.

```
Private Sub changefont()
Dim k As Controls 'procedure changes the font of labels once
enabled
For Each k In Me.Controls
If TypeOf k Is Label Then
k.FontBold = True
k.Underline = True
k.FontItalic = True
End If
```

Next

Functions

They are similar to procedures with one key difference that they return a value. This value can be assigned a variable or used in expression.

```
Function average(sum As Double, num As Integer)
    average = sum / num 'computes the average of num elements
End Function
```


RECORDSET

They are three types of record sets supported by visual basic

1. Tables
2. Dynaset
3. Snapshot

TABLE

A table (table type recordset) is a directly link to one of the physical tables stored in the database. this type of recordset provides the most direct link to the data. They are the only form of record sets that support indexes; hence searching a table for a specific record can be quicker than searching a dynaset or snapshot. when using tables data is addressed or modified from one table at a time, one record at a time

Advantages of using tables

You can use or create indexes to change the presentation order of the data in the tables during program execution.

You can perform rapid searches for an individual record using appropriate index and the seek command

Changes made to the tables by other concurrent users or programs are immediately available.

Disadvantages

You can't set filters on a table to limit the records being processed to a certain criteria.

You can't use find commands on a table (seek command finds only the first record that meets its criteria .

Using ADODC

```
Adodc1.CommandType = adCmdTable
```

```
Adodc1.RecordSource = "STUDENT"
```

```
Adodc1.Refresh
```

Using OLEDB CONNECTION

```
Set rctest = New ADODB.Recordset
```

```
rctest.Open "student", DCTEST, adOpenStatic, adLockOptimistic
```

Using DAO

```
Set rsmine = dbmine.OpenRecordset("STUDENT", dbOpenTable)
```

DYNASET

Is a group of information from one or more tables in a database .this information is composed of selected fields from the tables. Often presented in a specific order and filtered by a specific condition. they are updateable record sets and address the records present in the base tables at the time they are created. Although they do not automatically reflect additions or deletions of records made by users or programs after the dynaset is created. Dynaset is actually asset of record pointers that point to the specified data as it existed when the dynaset is created. {A dynaset is actually a set of record pointers that point to a specified data as it existed when the dynaset was created}.

Dynaset support information changes such as addition, edits and deletions.

Advantages of using dynasets

Dynasets enable you to join information from multiple tables

You can use find methods to locate or process every record meeting a specified criteria.

They enable limiting of the number of fields or records so are retrieved into the recordset.

They make use of filters and sort order properties to change the view of data.

You can't use indexes with dynaset hence you can't change the presentation order of the dynaset by changing the index or by creating a new one.

It does not automatically reflect additions or deletions

Using ADODC

Adodc1.CommandType = adCmdText

Adodc1.RecordSource = "SELECT * FROM STUDENT"

Adodc1.Refresh

Using OLEDB CONNECTION

Set rctest = New ADODB.Recordset

rctest.Open "select * from student", DCTEST, adOpenStatic,
adLockOptimistic

Using DAO

Set rsmine = dbmine.OpenRecordset("SELECT * FROM STUDENT",
dbOpenDynaset)

SNAPSHOTS

As the name implies, it is a "picture", or copy, of the data in a recordset at a particular point in time. They are similar to a dynaset in that it is created from base tables, using sql statements or from aquerydefinition, dynaset or another snapshots. the difference of dynaset to a snapshot whenever you want a set of data that isn't time sensitive. (it does not matter if the records are updated after creation of the snapshot.

Advantages

You can join information from multiple tables

You can use the find method

Record navigation and record set creation can be faster

Disadvantage

They are not updatable or indexes

Using ADODC

Adodc1.CommandType = adCmdStoredProc

Adodc1.RecordSource = "SELECT * FROM STUDENT"

Adodc1.Refresh

Using DAO

Set rsmine = dbmine.OpenRecordset("SELECT * FROM STUDENT", dbOpenSnapShot)

ODBC (OPEN DATABASE CONNECTIVITY)

Is a component of Microsoft's windows open system architecture (WOSA). ODBC provides asset of application program interface (API) functions, which makes it easier for a developer to connect to a wide range of database formats. ODBC makes use of SQL server, oracle, and inter base server.

You can connect to the database using odbc in the control panel, Odbc 32 bit.

Using code

We connect ODBC making use of the statement

```
DBEngine.RegisterDatabase databasename, driver, silent, attributes
```

Database name

A user definable string expression that specifies the data source name (for example, "my database")

Driver

A string expression that indicates the installed driver's name (for example oracle) as listed in the ODBC drivers' tab of the ODBC administrator.

Silent

It can be indicated as either true or false.

True specifies that the next parameter (attributes) indicate all connection information. Refer at fig 1

False specifies to display the driver setup dialog box and ignores the contents of the attributes parameter refer to fig 2.

Attributes

All connection information for using the ODBC driver .This parameter is ignored if silent is set to false.

When silent is true the program connects directly odbc the database trialedddy.mdb



Fig1

The code for the above form

```
Dim ssetting As String
Dim sdirect As String

Private Sub Command1_Click()
    ssetting = App.Path & "\trialeddy.mdb"
    sdirect = "Microsoft Access Driver (*.mdb)"
    DBEngine.RegisterDatabase "mydatabase", sdirect, True, ssetting
    MsgBox "we have made an odbc connection to the database"
End Sub
```

When silent is false we the program will generate an odbc setup dialogue that you will use to connect the database.

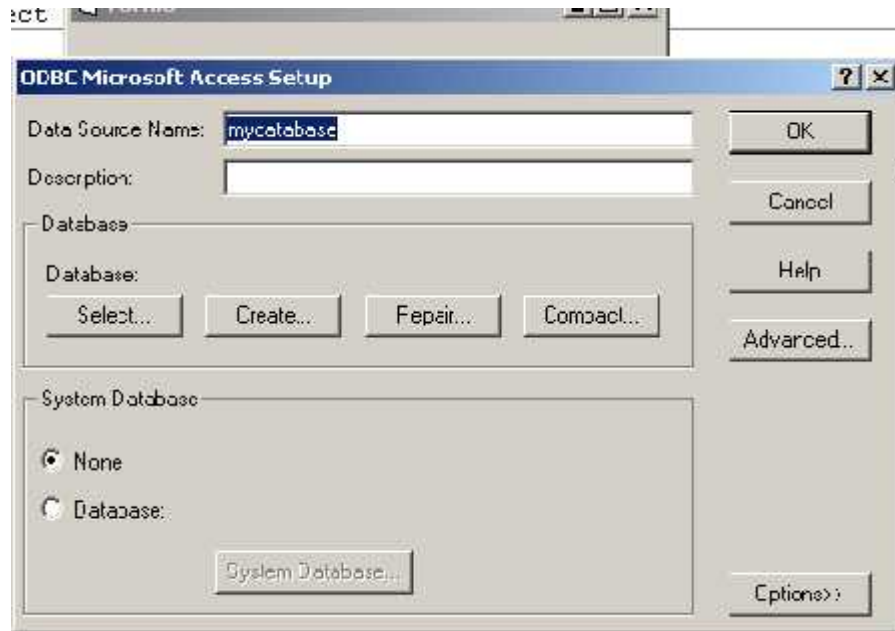


Fig2

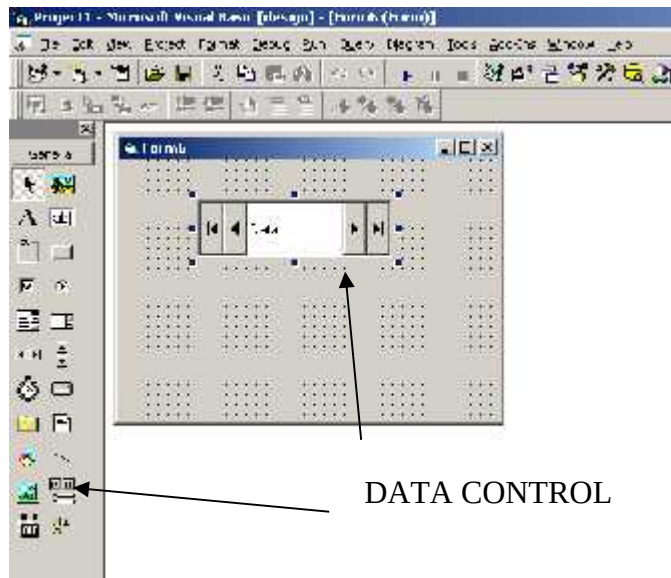
Section of the code necessary for generating the above dialogue

```
ssetting = App.Path & "\trialeddy.mdb"
sdirect = "Microsoft Access Driver (*.mdb)"
DBEngine.RegisterDatabase "mydatabase", sdirect, False, ssetting
MsgBox "we have made an odbc connection to the database"
```

TYPES OF DATA CONTROL OBJECTS

- I.) the data control and data bound controls
- II.) data access objects
- III.) remote data objects
- IV.) activex data objects

DATA CONTROL



Is a link between information in your database and the bound controls that you use to display the information. to use a data control you must

- 1.) set the database to connect to from the properties
- 2.) Select the part of the database to access.

By default the data control creates a dynaset_type recordset from one or more of the tables in your database.

Steps to follow when setting up a data control

- 1.) select the data control on your form
- 2.) draw the control on your form
- 3.) set the databasename property control
- 4.) set the recordsource property control

using code

```
data1.databasename=app. path & "\students.mbd"
                        =<data base name with full path>
data1.recordsource= "select * from <tablename>";
```

Data control is quick and easy but it lacks a few capabilities that are necessary for must data entry applications specifically adding and deleting records.

- 1.) adding records_


```
data1.recordset.addnew
```

 it allows one to enter new record, after a blank record is prepared in the copy buffer, which is a special place where records are edited before actually being entered into the database (you edit a new record).
- 2.) edit mode_ allows you to change the information in record.
- 3.) update_ commits the data in the copy buffer to the database when method is evoked.

DATA ACCESS OBJECTS (DAO)

is a more complex data use method where program commands are used and acts as an internal representation of physical data, data stored in some type of database or data management engine. the key reason for using program commands is the flexibility they give you beyond what is available with the data control. It allows you to perform more detailed input validation that is possible with built-in database engine rules because program commands do not directly access the database.

To use DAO

Select one of the Microsoft DAO libraries from references from the project name dialog box to create a database object in your program,.

to create a data base object in your program you open an existing data base

```

dim dbmine as database
Private sub form_load()
    set dbmine =DBEngine.workspace(0).OpenDatabase("<database
        name plus full path>")
    or
    set dbmine=OpenDatabase("<database name plus full path>")
End sub

```

DAO allows use of tables, dynasets, and snapshots when creating or using a recordset to access an entire table, specific fields and records from the table, or a combination of records and fields from multiple tables.

SAMPLE CODE FOR THE FORM

```

Dim dbmine As Database
Dim rsmine As Recordset, sP As String

Private Sub Command1_Click()
    With rsmine
        If .EOF = False And .BOF = False Then
            .MoveNext
            RETRIEVE
        Else
            .MoveFirst
        End If
    End With
End Sub

```

```
Private Sub Command10_Click()  
    Form1.Refresh  
End Sub  
  
Private Sub Command2_Click()  
    With rsmine  
  
        If .EOF = False And .BOF = False Then  
            .MovePrevious  
            RETRIEVE  
        Else  
            .MoveLast  
        End If  
    End With  
End Sub  
  
Private Sub Command3_Click()  
    With rsmine  
  
        If .EOF = False And .BOF = False Then  
            .MoveFirst  
            RETRIEVE  
        End If  
    End With  
End Sub  
  
Private Sub Command4_Click()  
    With rsmine  
  
        If .EOF = False And .BOF = False Then  
            .MoveLast  
            RETRIEVE  
        End If  
    End With  
End Sub  
  
Private Sub Command5_Click()  
    rsmine.AddNew  
    clear  
End Sub  
  
Private Sub Command6_Click()  
    SUBMITT  
    rsmine.Update  
    rsmine.MoveFirst  
    RETRIEVE  
End Sub  
  
Private Sub Command7_Click()  
    rsmine.Edit  
End Sub
```

Deleting records from the database

```
Private Sub Command8_Click()
    rsmine.Delete
    clear
End Sub
```

```
Private Sub Command9_Click()
```

Closing the record set and the database

```
    rsmine.Close
    Unload Me
End Sub
```

```
Private Sub Form_Load()
```

Assigning caption using code

```
    Command1.Caption = ">"
    Command2.Caption = "<"
    Command3.Caption = "<<"
    Command4.Caption = ">>"
    Command5.Caption = "ADDNEW"
    Command6.Caption = "SAVE"
    Command7.Caption = "EDIT"
    Command8.Caption = "DELETE"
    Command9.Caption = "CLOSE"
    Command10.Caption = "REFRESH"
```

Opening my database

```
sP = App.Path & "\trialeddy.MDB"
Set dbmine = DBEngine.Workspaces(0).OpenDatabase(sP)
Set rsmine = dbmine.OpenRecordset("STUDENT", dbOpenTable)
```

```
    RETRIEVE
```

Assigning the label caption the name of database table fields extracted

```
Label1.Caption = rsmine.Fields(0).Name
Label2.Caption = rsmine.Fields(1).Name
Label3.Caption = rsmine.Fields(2).Name
```

```
End Sub
```

Procedure to retrieve data from database fields and assign them to textboxes

```
Private Sub RETRIEVE()
    If rsmine.EOF = False And rsmine.BOF = False Then
        Text1.Text = rsmine!STUDENTID
        Text2.Text = rsmine!STUDENT_NAME
        Text3.Text = rsmine!STUDENT_AGE
    End If
End Sub
```


Procedure that allocates the database entity fields with textboxes details

```

Private Sub SUBMITT()
If rsmine.EOF = False And rsmine.BOF = False Then
    rsmine!STUDENTID = Text1.Text
    rsmine!STUDENT_NAME = Text2.Text
    rsmine!STUDENT_AGE = Text3.Text
End If
End Sub

```

Clearing procedure that clears all textboxes controls

```

Private Sub clear()
Dim myclear As Control
For Each myclear In Form2.Controls
    If TypeOf myclear Is TextBox Then
        myclear.Text = ""
    End If
Next myclear
End Sub

```

RDO (REMOTE DATA OBJECT)

Accessing data stored in a variety of remote locations can be done through the use of remote data objects(RDO).RDO are similar to DAO in that they make use of the same statement used in DAO to access data in the data bases.

RDOA enables conversion of programs from pc databases to client server databases much easier

ADO (ACTIVEX DATA OBJECTS)

It is used for controlling data as other controls but it has more flexibility in this regard than DAO

Example of ado's are:

ADODB (activex data object database)

Before using adodb you have to set one of activex controls in the references {project menu ,references then choose one of activex references from the dialog}

```

Dim DCTEST As ADODB.Connection
Dim rctest As ADODB.Recordset

```

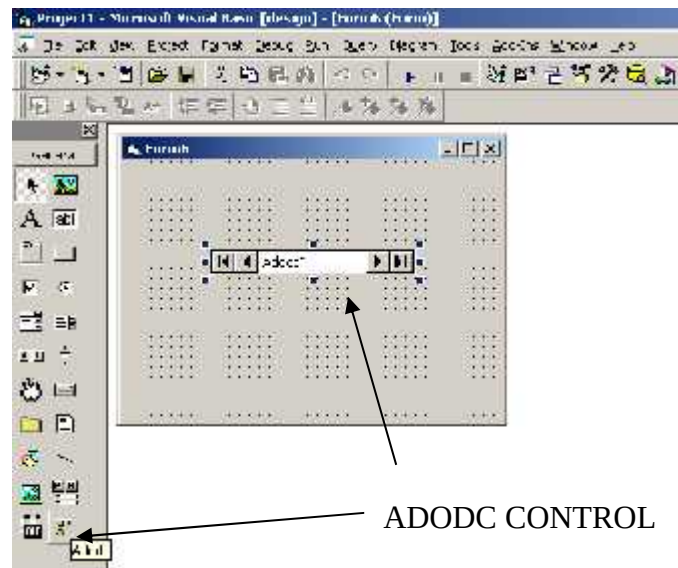
```

Private Sub Form_Load()
Set DCTEST = New ADODB.Connection
DCTEST.CursorLocation = adUseClient
DCTEST.Provider = "Microsoft.Jet.OLEDB.4.0;"
DCTEST.ConnectionString = App.Path & "\trIAleddy.mdb" 'LOADING THE DATABASE
DCTEST.Open 'OPEN DATABASE TO ENAMBLE ACCESSIBILITY
Set rctest = New ADODB.Recordset

    rctest.Open "select * from student", DCTEST, adOpenStatic, adLockOptimistic
End sub

```

ADODC (activex data object data control)



It is a new control that works like data control

To use ADODC, you must add it to the visual basic toolbox. To do so, right-click in an empty area of the tool box, select components from the context menu, and add the Microsoft ADODC 6.0 control.

Making The Connection To Database

ADODC has three methods of building a string connection to a database.

1.) Use data link file

This option allows you to load saved connection information from an MDL (Microsoft data link file).

2.) Use ODBC data source name

Here, you can select from one of the ODBC DSNs set up on your system. You can also click the new button to create a new DSN, just as you would from the control panel.

3.) Use connection string

This option allows you to specify a connection string directly. Clicking the build button rings up a wizard that helps you build the connection string.

Connecting recordsource

Record source informs visual basic exactly what data to retrieve; you do so by selecting the record source tab of the property pages dialogue box.

ADODC has four types of commands to retrieve data:

Adcmdtext

Runs an SQL query on the datasource

Adcmdstoredproc

Calls a stored procedure on the database server

Adcmdtable

Specifies the name of the database table; it is used to return the entire table.

Adcmdunknown

For unknown recordset type

Connecting the database by setting the connection string property

```
Adodc1.ConnectionString =
"DBQ=C:\NOTESEDDY\EDDY\TRIALEDDY.mdb;DefaultDir=C:\NOTESEDDY\EDDY;Driver={Microsoft Access Driver (*.mdb)};DriverId=25;FIL=MS
Access;FILEDSN=C:\NOTESEDDY\EDDY\TRIALEDDY.mdb.dsn;MaxBufferSize=2048;Max
ScanRows=8;PageTimeout=5;SafeTransactions=0;Threads=3;UID=admin;UserCommitSync=Yes;"
```

Setting the command type

```
Adodc1.CommandType = adCmdText
```

Setting record source property

```
Adodc1.RecordSource = "SELECT * FROM STUDENT"
```

Executing the refresh method of the ADO DC to retrieve the data.

```
Adodc1.Refresh
```

CREATING REPORTS

- 1.) Create reports using **PRINT REPORTING FUNCTION** and other graphics methods. (Use of third party reporting addin)

- o It is time wastage and also wastes effort
- o Additional costs while using report addin and DLL distribution requirement.

2.) USING THE DATA REPORT

Enable display of a print preview screen, with print and export buttons, from an ADO data source.

It is also known as an activex designer

The report has five main sections

Report header	- information is displayed only once at the top of the report.
Page header	- information is displayed at the top of every page.
Detail section	-This section or part of the report is repeated for every or each record in the ADO data source.
Page footer	-Information displayed at the bottom of every page
Report footer	-Information displayed at the end of report.

Data report controls

Rpt label	-specifies a label used to display text that is not data bound, such as a column name or date printed.
Rpt textbox	- displays the contents of a database field. (Only used in the detail section).
Rpt image	- contains a picture or other graphic image, such as accompany logo.

Rpt line	-Enables you to draw lines on the report _for example to separate the different sections of the report
Rpt shape	-Enables you to draw a variety of shapes on the control to highlight information or provide other visual effects.
Rpt function	-Allows you to place a field in the header or footer that contains one of several simple math functions, such as a total of all the other values in a particular data field.

To create a sample data report (steps)

- 1.) set up the ADO record set to be used by the report
- 2.) add a data report object to vb project
- 3.) Design the report by adding fields on the data report designer form.
- 4.) Write code to display the report in the program.

- setting up the data source to be used by the report.
 - your can make use of data environment
 - code

- 1.) Add the reference for Microsoft activex data objects to your project.
- 2.) set up the database ODBC data source
- 3.) Declare form-level variables, e.g. cn and rs , to represent ADO connection and record set objects respectively.

```

dim cn as ADODB.connection
dim rs as ADODB.recordset

private sub form_load()
    set cn =new ADODB.connection
    cn.open "DSN=DatabaseName"
    set rs =cn.execute("select * from <tablename>")
end sub

private sub form_unload()
    rs.close()
    cn.close()
end sub

4.) to show the report type in the show command button procedure
private sub cmdshow_click()
    set datareport1.datasources=rs
    datareport1. show
end sub

```

3.) USING CRYSTAL REPORT

Is a third party reporting package included with vb. (it allows you to create reports from many different types of data sources). You can design and test reports graphically by using the crystal reports designer. The report layout and other information are stored in a custom crystal reports file format with .RPT extension.

Methods do display reports

- I.) Use the crystal reports control that provides an easy way to integrate the finished reports into your vb program. e.g. crystalreport1.printreport.

- II.) Use crystal reports an API call, which allows you to access the crystal reports engine without using custom control. All the API calls for crystal reports are located in a file called GLOBAL32.BAS, which is installed with your crystal reports program directory.
- III.) The crystal reports web server; available in crystal reports professional edition, works with your web server to deliver reports to an internet client.

LINKING OR OPENING APPLICATIONS USING VISUAL BASIC

Visual basic can allow you to open applications such as word ,notepad,excel and other Microsoft and non Microsoft applications

OPENING EXCEL APPLICATION

```
Dim oApp As Object
```

```
Set oApp = CreateObject("Excel.Application")
```

```
oApp.Visible = True
```

```
Only XL 97 supports UserControl Property
```

```
oApp.UserControl = True
```

OPENING WORD APPLICATION

```
Dim oApp As Object
```

```
Set oApp = CreateObject("Word.Application")
```

```
oApp.Visible = True
```

OPENING NOTEPAD

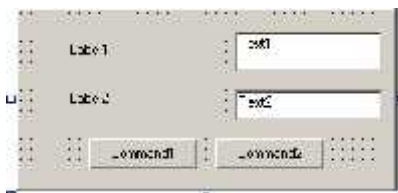
```
Call shell ("notepad.exe", 1)
```

Assignments

ASSIGNMENT 1.1

Login form

Label 1	text 1
Label 2	text 2
Command 1	command 2



- 1) Highlight the form and press F4 to get window properties.
 - a) Name the form –frmlogin
 - b) Caption the form –login
- 2) Highlight the label, text boxes and command buttons and name them.

	<u>names</u>	<u>caption</u>
label 1	lblusername	username
label2	lblpassword	password
text1	txtusername	text clear
text2`	txtpassword	text clear
command1	cmdokay	okay
command2	cmdcancel	cancel

- 1) **Double click the form or in the project explorer click code view or in the view menu click view code.**
- 2) **In the form procedure type**

```
Private sub form_load()
    Txt password.maxlenth=8
    Txt password.passwordclear="*"
End sub.
```

3. In the Okay button, type

```
Private sub cmdokey_click()
    If txtusername= "admin" and txtpassword.Text= "user" then
        MsgBox "correct logging"
    Else
        MsgBox "error on logging"
        Txtusername.text =""
        Txtpassword.text = ""
    End if
End sub
```

```
Private sub cmdcancel_click()
    Txtusername.text=""
    Txtpassword.text=""
End sub
```

ASSIGNMENT 1.2

Form calculations

Label 1	text 1						
Label 2	text 2						
Label 3	text 3						
Cmd1	cmd2	cmd3	cmd4	cmd5	cmd6	cmd7	cmd8

- 1) Name the form using window properties
Frmcalculator
Caption it as calculator

Name all labels ,textboxes,and commands as

Lblfirstnumber
Lblsecondnumber
Lblanswer

Txtfirstnumber
Txtsecondnumber
Txtanswer

Cmdplus
Cmdminus
Cmdmult
Cmddiv
Cmdintdiv
Cmdmodus
Cmdexpo
Cmdequals
Cmdclear

- 3) In procedure form load

```
Private sub form_load ()
    Call clear
    Lblfirstnumber.Caption="first number"
    Lblsecondnumber.caption=" second number"
    Lblanswer.caption="answer"
    Cmdplus.caption="+"
    Cmdminus.caption="-"
```

```

Cmdmult.caption="*"
Cmddiv.caption="/"
Cmdintdiv.caption="\
Cmdmodus.caption=" mod"
Cmdexpo.caption="^"
Cmdequal.Caption="="
Cmdclear.Caption="clr"
End sub

```

```

private sub cmdplus_click()

```

```

    txt answer=(txt firstnumber.text)+val(txt+second number.text)
end sub

```

repeat the same statement modified to fit the command operation name for the rest of the buttons.

3) Type the clear procedure

```

Private sub clear()
    Dim myclear as control
    For each myclear in me.controls
        If typeof myclear is textbox then
            Myclear.text=""
        End if
    Next
End sub

```

Assignment 1.3

Redo assignment 1.2 and now declare globally

```

Dim sign as string

```

In each command button procedure now type sign variable assigned an operator as shown below ,the event should be relevant to the procedure.
e.g. for addition

```

Private sub cmdplus_click()
    Sign="+"
End sub

```

Create an arithmetic function tht will do all the operational functions according to allocation

Function arithmetic (ssign as string, num1 as double, num2 as double)

```

    Select case ssign
        Case "+"
            Arithmetic =num1+num2
        Case "^"
            Arithmetic =num1 +num2
        {continue with the cases until all the operations are covered}
    end select
end function

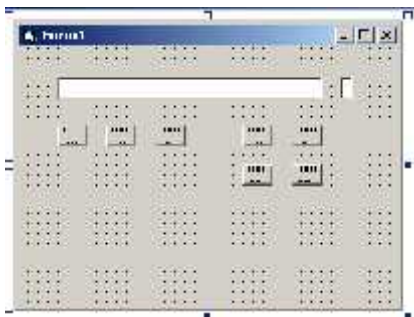
```


in the equals function type

```
private sub equals_click()
    txtanswer.text=arithmetic(sign,val(txtfirstnumber.text),val(txtsecondnumber.text))
end sub
```

ASSIGNMENT1.4

Complete the calculator below :



```
Dim num1 As Double, num2 As Double, psign As String, sign
```

```
Private Sub Command1_Click()
```

```
If Text1.Text = "0 " Then
```

```
    Text1.Text = Command1.Caption
```

```
ElseIf Text2.Text <> "" Then
```

```
    Text1.Text = Command1.Caption
```

```
    Text2.Text = ""
```

```
Else
```

```
    Text1.Text = Text1.Text & Command1.Caption
```

```
End If
```

```
End Sub
```

```
Private Sub Command3_Click()
```

```
    Text1.Text = Text1.Text & Command3.Caption
```

```
End Sub
```

```
Private Sub Command4_Click()
```

```
    Text1.Text = flsh(psign, num1, Text1)
```

```
    psign = Command4.Caption
```

```
    num1 = Val(Text1.Text)
```

```
    Text2.Text = psign
```

```
End Sub
```

```
Private Sub Command6_Click()
```

```

    Text1.Text = clear2(Text1)
    'Call clear
End Sub

```

```

Private Sub Command7_Click()
    num2 = Val(Text1.Text)
    If psign = "+" Then
        Text1.Text = num1 + num2
    End If

```

```

End Sub

```

```

Private Sub Form_Load()
    Command1.Caption = 1
    Command2.Caption = 2
    Command3.Caption = 3
    Command4.Caption = "+"
    Command5.Caption = "*"
    Command7.Caption = "="
    Command6.Caption = "cl"
    clear
End Sub

```

```

Private Sub clear()
    Dim myclear As Control
    For Each myclear In Form1.Controls
        If TypeOf myclear Is TextBox Then
            myclear.Text = "0"
            myclear.Alignment = 1 - rightjustify
        End If
    Next
End Sub

```

```

Private Function clear2(otext As Object)
    otext.Text = "0"
    clear2 = otext.Text
End Function

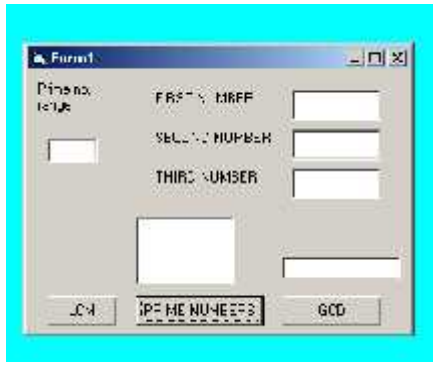
```

```

Private Function flsh(sign As String, num As Double, otext As Object)
    If sign = "+" Then
        flsh = num + Val(otext.Text)
    ElseIf sign = "-" Then
        If num <> 0 Then
            flsh = num - Val(otext.Text)
        Else
            flsh = Val(otext.Text)
        End If
    ElseIf sign = "*" Then
        If num <> 0 Then
            flsh = num * Val(otext.Text)
        Else
            flsh = Val(otext.Text)
        End If
    Else
        flsh = Val(otext.Text)
    End If
End Function

```

ASSIGNMENT 1.5



TEST THE ABOVE INTERFACE USING THE CODE BELOW THEN CONVERT THE PROCEDURES INTO RELEVANT FUNCTIONS AND PROCEDURES TO DO THE SAME WORK

```

Dim i As Integer, j As Integer, p As Integer
Dim PL(1 To 200000) As Double, tl As Integer
Private Sub Command1_Click()
    i = 1
    p = 1
    tl = 1
    While i <= Val(Text5.Text)

        For j = 2 To Val(Text5.Text)

            If i Mod j <> 0 And i <> j Then
                p = i
            ElseIf i = 2 Then
                p = 2
            ElseIf (i Mod j) = 0 And i <> j Then
                p = 0
                Exit For
            End If
        Next j
        If p <> 0 Then
            PL(tl) = p
            tl = tl + 1
        End If
        i = i + 1
    Wend
    For T% = 1 To tl - 1
        List1.AddItem PL(T)
    Next T
End Sub

Private Sub Command2_Click()
    Dim num1 As Integer, num2 As Integer, num3 As Integer
    Dim P1 As Double, nm As Integer, uj As Integer

```

```

    nm = tl - 1

num1 = Val(Text2.Text)
num2 = Val(Text3.Text)
num3 = Val(Text4.Text)
P1 = 1

uj = 2
While uj <= nm

If num1 Mod PL(uj) = 0 Then
    If num2 Mod PL(uj) = 0 Then
        If num3 Mod PL(uj) = 0 Then

            P1 = P1 * PL(uj)

            num1 = num1 / PL(uj)
            num2 = num2 / PL(uj)
            num3 = num3 / PL(uj)
        Else
            uj = uj + 1
        End If
    Else
        uj = uj + 1
    End If
Else
    uj = uj + 1
End If

Wend

Text1.Text = P1
End Sub

Private Sub Command3_Click()
Dim ll As Integer, nt As Integer, pll As Double
    ll = 2
    num1 = Text2.Text
    num2 = Text3.Text
    num3 = Text4.Text
    pll = 1
    While ll <= tl - 1
        If num1 Mod PL(ll) = 0 Then
            num1 = num1 / PL(ll)
            If num2 Mod PL(ll) = 0 Then
                num2 = num2 / PL(ll)
            End If
            If num3 Mod PL(ll) = 0 Then
                num3 = num3 / PL(ll)
            End If

            nt = nt + 1
        ElseIf num2 Mod PL(ll) = 0 Then

```

```
num2 = num2 / PL(ll)
If num1 Mod PL(ll) = 0 Then
    num1 = num1 / PL(ll)
End If
    If num3 Mod PL(ll) = 0 Then
        num3 = num3 / PL(ll)
    End If

    nt = nt + 1
ElseIf num3 Mod PL(ll) = 0 Then
    num3 = num3 / PL(ll)
    If num2 Mod PL(ll) = 0 Then
        num2 = num2 / PL(ll)
    End If
        If num1 Mod PL(ll) = 0 Then
            num1 = num1 / PL(ll)
        End If
        nt = nt + 1

Else

    ll = ll + 1
End If
If nt <> 0 Then
    For d% = 1 To nt
        pll = pll * PL(ll)
    Next d
End If
    nt = 0
Wend
Text1.Text = pll
End Sub
```